

Краткосрочное прогнозирование событий в эгоцентрическом видео по извлечённым RGB-признакам

EPIC-KITCHENS-55 / RULSTM features, next-action anticipation

Сулиман Адам

21 мая 2026 г.

Формулировка

По короткой наблюдаемой последовательности визуальных признаков нужно предсказать следующее действие пользователя в эгоцентрическом видео.

$$X = (f_1, \dots, f_T), \quad f_t \in \mathbb{R}^{1024}$$

$$y = (v, n), \quad v = \text{verb}, \quad n = \text{noun}$$

- Вход: последовательность RGB-признаков до следующего действия.
- Выход: глагол, существительное и совместная пара (v, n) .
- Цель: получить эффективную модель для ограниченных вычислительных ресурсов.

Мотивация и ограничения

- Полное обучение на raw video требует дорогого backbone и большого количества времени и ресурсов.
- Доступная среда: Kaggle kernel, до 2x T4, около 30 GiB RAM, ограниченное время.
- Основной эксперимент: **feature-based anticipation**, не требует обучения визуального backbone, всё ещё давая хорошие результаты в egocentric action anticipation.
- Мы не обучаем визуальный backbone.

Исследовательский вопрос

Можно ли улучшить краткосрочное прогнозирование следующего действия, используя компактную temporal модель поверх фиксированных RGB-признаков?

- Использован набор: **EPIC-KITCHENS-55 RULSTM RGB pre-extracted features**.
- Это не raw video, а предвычисленные признаки из RGB-потока.
- Каждый sample имеет форму:
$$14 \times 1024$$
- 14 временных фрагментов перед действием, 1024 признака на фрагмент.

Параметр	Кол-во
Train samples	23,472
Validation samples	4,972
Verb classes	119
Noun classes	321
Seen train actions	2,288

Что такое RGB-признаки

- RGB-признаки — это численные векторы, заранее извлеченные из обычных цветных кадров.
- В RULSTM эти признаки извлечены с помощью **BN-Inception CNN**, обученной для egocentric action recognition в схеме **Temporal Segment Networks (TSN)**.
- Используется только RGB stream; flow/object признаки в основных экспериментах не используются.
- Схема получения признаков:

$$\text{RGB frames} \rightarrow \text{TSN BN-Inception} \rightarrow \mathbf{f}_t \in \mathbb{R}^{1024}$$

- Наша работа начинается после этого этапа:

$$(\mathbf{f}_1, \dots, \mathbf{f}_{14}) \rightarrow \text{temporal model} \rightarrow (v, n)$$

- Поэтому вклад проекта — не feature extraction, а эффективное temporal/action-level modeling поверх фиксированных RGB features.

Top-k accuracy

Для каждого validation sample модель ранжирует возможные ответы по score. Top-k засчитывается как 1, если правильный ответ входит в первые k кандидатов.

- **Verb top-1 / top-5:** правильно ли предсказан глагол, например *open, take, put*.
- **Noun top-1 / top-5:** правильно ли предсказан объект действия, например *drawer, knife, plate*.
- **Joint action top-1 / top-5:** правильно ли предсказана вся пара (v, n) . Это главная метрика: для next-action anticipation важно угадать не только действие, но и объект.

Joint action top-k считается по матрице всех verb-noun пар:

$$S_{v,n} = s_v + s_n + \lambda \log P_{\text{train}}(v, n) + \gamma s_{a(v,n)}$$

Здесь s_v, s_n — logits отдельных голов, $\log P_{\text{train}}$ — статистический prior, s_a — score action-head для seen action pairs.

- **Число обучаемых параметров:** размер temporal model без frozen RGB backbone. Показывает, насколько модель компактна.
- **Время обучения на эпоху:** wall-clock время одного прохода по train split.
- **Inference time per validation sample:** среднее время предсказания на один validation sample. Это важно для сценария online anticipation.
- **Peak GPU memory:** максимальная выделенная CUDA-память во время запуска.

Почему качество и эффективность показываются вместе

Цель проекта — не максимизация accuracy любой ценой, а хороший компромисс: улучшить joint action top-5 при малом числе параметров, быстром обучении и небольшом расходе GPU memory.

Сравниваемые методы

- 1 **Prior-only**: не использует X , а ранжирует пары по сглаженной частоте $P_{\text{train}}(v, n)$. Это нижняя граница “только статистика labels”.
- 2 **MeanPool MLP**: заменяет последовательность средним вектором $\bar{f} = \frac{1}{T} \sum_t f_t$. Проверяет, достаточно ли общего визуального контекста без порядка кадров.
- 3 **GRU**: читает признаки по времени и хранит скрытое состояние h_t . Проверяет вклад temporal order и динамики перед действием.
- 4 **TinyRMT / AttentiveGRU**: добавляют память или attention поверх короткой последовательности. Проверяют, помогает ли выбор важных временных фрагментов.
- 5 **JointGRU**: GRU + auxiliary head для seen verb-noun pairs. Добавляет прямой score для joint action, а не только сумму verb и noun logits.

Логика сравнения

Идем от статистического baseline к временной модели и затем к модели, которая явно учитывает совместимость verb-noun пары.

Предложенный метод: JointGRU

- На шаге t GRU получает текущий RGB-признак f_t и прошлую память h_{t-1} .
- **Update gate** z_t решает, сколько новой информации записать.
- **Reset gate** r_t решает, какую часть прошлой памяти использовать.
- После последнего шага h_T — компактное описание наблюдаемого контекста перед следующим действием.
- Из h_T строятся три головы:
 - verb head;
 - noun head;
 - seen-action head.

$$z_t = \sigma(W_z f_t + U_z h_{t-1})$$

$$r_t = \sigma(W_r f_t + U_r h_{t-1})$$

$$\tilde{h}_t = \tanh(W_h f_t + U_h (r_t \odot h_{t-1}))$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

$$s_v = W_v h_T, \quad s_n = W_n h_T, \quad s_a = W_a h_T$$

$$S_{v,n} = s_v(v) + s_n(n) + \lambda \log P(v, n) + \gamma s_a(v, n)$$

Action head обучается только на парах (v, n) , встреченных в train; для unseen пар остается сумма verb/noun score и prior.

$$\mathcal{L} = \text{CE}(\mathbf{s}_v, v) + \text{CE}(\mathbf{s}_n, n) + \beta \text{CE}(\mathbf{s}_a, a)$$

$$\text{CE}(\mathbf{s}, y) = -\log \frac{\exp(\mathbf{s}_y)}{\sum_j \exp(\mathbf{s}_j)}$$

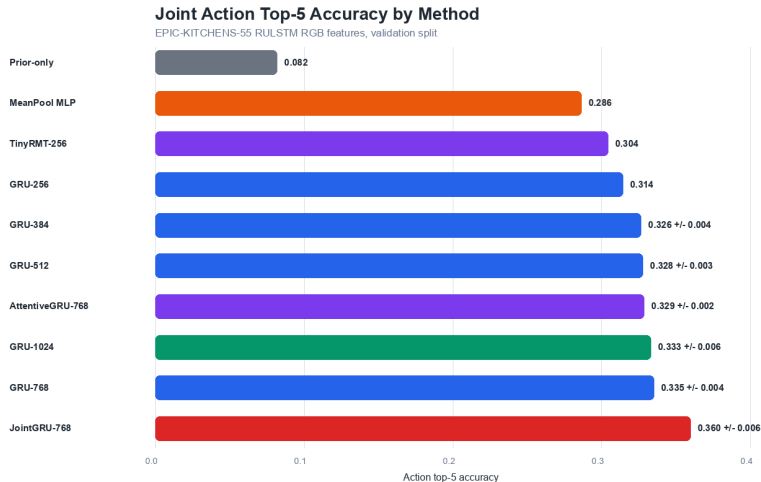
- $\mathbf{s}_v \in \mathbb{R}^{119}$: logits по классам глаголов, v — истинный verb label.
- $\mathbf{s}_n \in \mathbb{R}^{321}$: logits по классам существительных, n — истинный noun label.
- $\mathbf{s}_a \in \mathbb{R}^{2288}$: logits по verb-noun парам, встреченным в train; a — индекс истинной пары.
- Cross-entropy штрафует модель, если logit истинного класса мал относительно остальных logits.
- β управляет вкладом auxiliary action head: для обычных baseline $\beta = 0$, для JointGRU $\beta = 0.3$.

Основные результаты

Метод	Params	ms/sample	Action@1	Action@5
Prior-only	0	0.164	0.026	0.082
MeanPool MLP	1.01M	0.103	0.111	0.286
TinyRMT-256	0.91M	0.100	0.142	0.304
GRU-384	1.79M	0.105	0.154	0.326
GRU-512	2.59M	0.099	0.156	0.328
GRU-768	4.47M	0.102	0.158	0.335
JointGRU-768	6.23M	0.113	0.168	0.360

Лучший результат: JointGRU-768, $\lambda = 0.1, \gamma = 2.0$, Action@5 = 0.360 ± 0.006 .

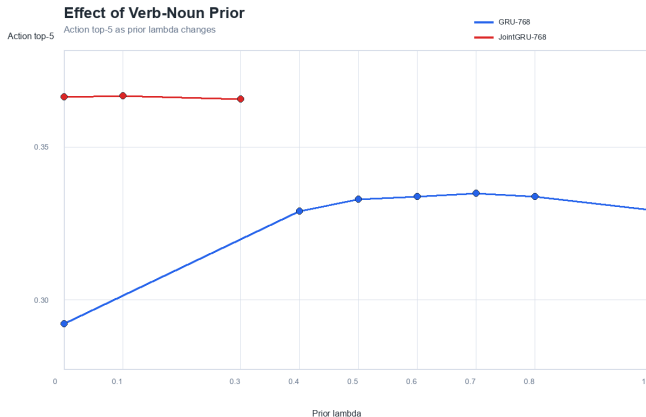
Качество: Action top-5



Качество против эффективности

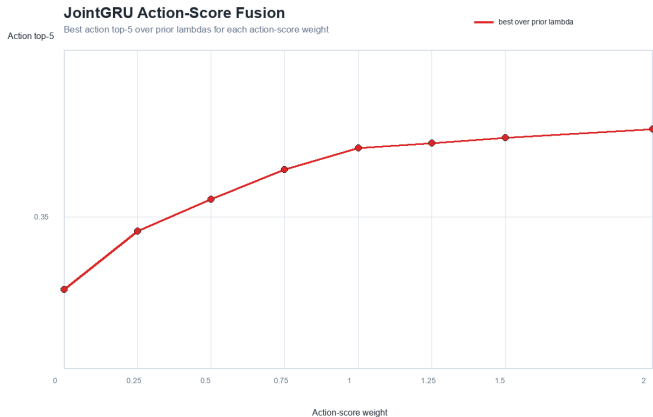


Ablation: train prior



Prior заметно помогает обычному GRU, но для JointGRU основную роль играет action head.

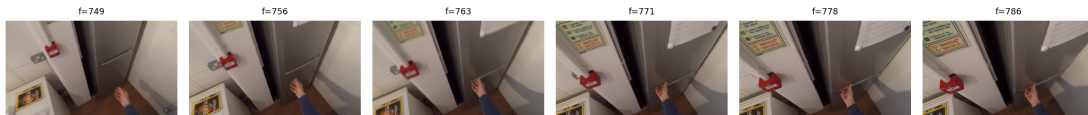
Ablation: action-score fusion



Увеличение веса action head улучшает совместное предсказание до $\gamma = 2.0$, но прирост после $\gamma = 1.0$ небольшой.

Качественный пример: validation sample 10

P01_01 sample 10: GT= close_drawer, top1= close_drawer



Ранг	Предсказанная пара	Комментарий
1	close_drawer	совпадает с ground truth
2	open_drawer	близкое действие с тем же объектом
3	close_door	похожий verb, другой noun
5	take_drawer	тот же объект, другой verb

Кадры используются только для визуализации; предсказание сделано по соответствующим RULSTM RGB-признакам.

Интерпретация результатов

- Статистика train labels сама по себе слаба: Prior-only Action@5 = 0.082.
- MeanPool уже дает разумный baseline: Action@5 = 0.286.
- GRU улучшает качество за счет временного порядка: Action@5 = 0.335.
- Простое увеличение hidden size до 1024 не помогает.
- Attention/TinyRMT не превзошли plain GRU.
- JointGRU улучшает качество за счет дополнительной action-level supervision.

Главный вывод

При фиксированных RGB-признаках важен не только temporal encoder, но и согласование factorized verb/noun prediction с совместной action pair prediction.

Ограничения

- Модель не обучается на raw video frames.
- RGB backbone фиксирован и не дообучается.
- Используются только RGB-признаки: без optical flow, audio, object detections.
- Результаты являются validation performance, не test-server performance.
- Hyperparameters подбирались по validation set, поэтому validation не является полностью untouched final test.

Почему это все равно корректно

Задача и labels реальные: EPIC-KITCHENS-55 validation action anticipation.

Ограничение касается только того, что визуальный backbone уже предвычислен.

- Подготовлен Kaggle-compatible экспериментальный pipeline.
- Реализованы data discovery, feature dataset loader и synthetic fallback.
- Реализованы baselines: Prior-only, MeanPool MLP, GRU, TinyRMT/AttentiveGRU.
- Разработан и проверен JointGRU с auxiliary seen-action head.
- Реализованы joint action top-k metrics и efficiency metrics.
- Проведены seed sweeps, ablations и анализ результатов.
- Подготовлены финальные таблицы и графики для презентации.

Репозиторий и воспроизводимость

- Основной notebook:

`video_event_prediction_rgb_features_experiments.ipynb`

- VS Code Python notebook:

`video_event_prediction_rgb_features_experiments.py`

- Скрипт графиков:

`results/make_presentation_figures.py`

- Результаты:

`results/final_results_table.csv`

TODO: вставить ссылку на GitHub/Kaggle notebook перед сдачей

- Построен компактный pipeline для next-action anticipation по RGB-признакам.
- Получено сравнение аналитического baseline и нескольких нейросетевых методов.
- Лучший метод: JointGRU-768 с auxiliary action head.
- Улучшение над сильным GRU-768 baseline:

0.335 $\rightarrow$ 0.360 Action@5

- Метод остается легким: 6.23M параметров и около 0.11 ms/sample на validation inference.

Финальный вывод

В условиях ограниченных ресурсов полезнее улучшать temporal/action-level modeling поверх предвычисленных признаков, чем пытаться обучать тяжелый raw-video pipeline.